AG TECHNOLOGIES

Architecture de Services Partagés

Cahier des Charges Technique

Service de Paiement Centralisé

Payment Service — AGT-Payment

Version	1.2
Date	Avril 2026
Statut	Implementation-ready
Classification	Confidentiel
Auteur	Équipe Architecture
Dépendances	Auth v2.1, Users v1.0, Abonnement v1.2, Notification v1.2

Document confidentiel — Réservé à l'usage interne d'AG Technologies

Historique des Révisions

Ver.	Date	Auteur	Description
1.0	2026-04-03	Équipe Archi.	Création initiale. Exécutant pur (event-driven). Providers : Orange Money, MTN MoMo, Stripe, PayPal. Webhooks normalisés. Idempotence. Réconciliation. Contrats inter-services formalisés (Abonnement, Wallet, Notification). Diagramme PlantUML en annexe.
1.1	2026-04-04	Équipe Archi.	Post-audit : machine à états formalisée (transitions autorisées). Modèle `PaymentMethod` abstrait. Sécurité webhooks détaillée (HMAC, IP whitelist, replay protection, timestamp validation). Timeout/expiration par provider. Idempotence renforcée (visibilité contractuelle).
1.2	2026-04-04	Équipe Archi.	Version finale : statut *Implementation-ready*. Matrice d'autorisation avec chemins complets. Sémantique PaymentMethod corrigée (`external_wallet` pour PayPal). Propagation complète du statut `cancelled` (normalisation, événements, admin).

Table des matières

1 Introduction **4**
1.1 Périmètre . . . 4
1.2 Hors périmètre . . . 4

2 Contexte & Problématique **4**
2.1 Contexte . . . 4
2.2 Problématique . . . 4

3 Vision & Objectifs **5**
3.1 Vision . . . 5
3.2 Objectifs . . . 5

4 Parties Prenantes & Utilisateurs **5**

5 Besoins Fonctionnels **5**
5.1 Initiation de Paiement . . . 6
5.2 Webhooks & Confirmation . . . 6
5.3 Gestion des Providers . . . 7
5.4 Transactions & Traçabilité . . . 8
5.5 Réconciliation . . . 8
5.6 Administration . . . 9

6 Besoins Non-Fonctionnels **9**

7 Architecture Technique **9**
7.1 Vue d'ensemble . . . 9
7.2 Stack Technique . . . 10
7.3 Dépendances & Règles d'Architecture . . . 10
7.4 Stratégie de résilience . . . 11
7.5 Diagramme d'architecture . . . 11

8 Modélisation **12**
8.1 Diagramme de Classes . . . 12
8.2 MLD — Modèle Logique de Données . . . 12
8.2.1 Table `transactions` . . . 13
8.2.2 Table `transaction_status_history` . . . 14
8.2.3 Table `webhook_logs` . . . 14
8.2.4 Table `provider_configs` . . . 15
8.2.5 Table `platform_payment_configs` . . . 15
8.2.6 Table `reconciliation_reports` . . . 16

9 Contrats d'API **16**
9.1 Health Check . . . 16
9.1.1 GET /payments/health . . . 16
9.2 Initiation de Paiement . . . 16
9.2.1 POST /payments/initiate . . . 16
9.2.2 GET /payments/{transactionId} . . . 18
9.2.3 GET /payments . . . 18
9.2.4 POST /payments/{transactionId}/cancel (v1.1) . . . 18
9.3 Webhooks Providers . . . 18
9.3.1 POST /payments/webhooks/orange-money . . . 18

9.3.2 POST /payments/webhooks/mtn-momo . . . 18
9.3.3 POST /payments/webhooks/stripe . . . 19
9.3.4 POST /payments/webhooks/paypal . . . 19
9.4 Configuration Providers . . . 19
9.4.1 POST /payments/providers . . . 19
9.4.2 PUT /payments/providers/{provider} . . . 20
9.4.3 PUT /payments/platforms/{platformId}/providers . . . 20
9.5 Administration . . . 20
9.5.1 GET /payments/admin/stats . . . 20
9.5.2 POST /payments/admin/{transactionId}/force-status . . . 20
9.5.3 GET /payments/admin/reconciliation . . . 21
9.5.4 DELETE /payments/by-user/{userId} . . . 21

10 Contrats Inter-Services 21
10.1 Paiement → Auth (JWT utilisateur) . . . 21
10.2 Paiement → Auth (token S2S) . . . 21
10.3 Abonnement → Paiement (événements RabbitMQ entrants) . . . 21
10.4 Paiement → Abonnement / Wallet (événements RabbitMQ sortants) . . . 22
10.5 Paiement → Notification (alertes) . . . 22
10.6 Users → Paiement (Purge RGPD) . . . 22

11 Matrice d'Autorisation 23

12 Recommandations & Évolutions 23

1 Introduction

Ce document constitue le cahier des charges du **Service de Paiement Centralisé** (ci-après « Payment Service ») destiné à l'écosystème AGT. Il sert de référence unique pour la conception, le développement et les décisions techniques.

1.1 Périmètre

Le service Paiement couvre l'**exécution des transactions financières** : initiation de paiements via providers externes, réception et normalisation des webhooks, confirmation ou échec des transactions, traçabilité complète et réconciliation.

Principe fondamental

Paiement est un **exécutant pur**. Il ne décide pas *quoi* facturer ni *combien* : c'est le service demandeur (Abonnement, Wallet, ou une plateforme) qui envoie un ordre de paiement. Paiement l'exécute et confirme le résultat.

1.2 Hors périmètre

- Authentification (déléguée au Service Auth v2.1)
- Gestion des abonnements, plans, quotas (Service Abonnement v1.2)
- Répartition des fonds, commissions, soldes (futur Service Wallet)
- Profils utilisateur (Service Users v1.0)
- Paiement de courses temps réel MboaMove (géré par MboaMove)
- Remboursements (phase 2)
- Logique métier propre aux plateformes

Convention d'identité

Dans tout ce document :
- `user_id` désigne `users_auth.id` (= `sub` du JWT)
- `platform_id` désigne un UUID du registre Auth (`platforms.id`)
- Paiement ne stocke aucune table locale de plateformes ni d'utilisateurs.

2 Contexte & Problématique

2.1 Contexte

Plateforme	Statut	Besoin paiement
AGT-Bot	En développement	Paiement d'abonnements (plans Pro, Enterprise)
AGT-Market	En développement	Paiement de produits/services, commissions vendeurs
MboaMove	En développement	Cash-in/cash-out wallet chauffeurs et passagers
SALMA	Migration prévue	Paiement de formations et certifications
Futures	Planifié	Toute plateforme nécessitant un encaissement

2.2 Problématique

Sans service centralisé, chaque plateforme intégrerait ses propres providers de paiement, entraînant : duplication des intégrations (Orange Money, Stripe, etc.), incohérence dans la gestion des webhooks, absence de réconciliation unifiée, risques de sécurité liés à la manipulation de données financières, et impossibilité de tracer les transactions à travers l'écosystème.

3 Vision & Objectifs

3.1 Vision

Un service **générique et abstrait** qui encapsule tous les providers de paiement. Les plateformes et services internes ne parlent jamais directement à Orange Money, Stripe ou PayPal : ils envoient un ordre de paiement au Payment Service, qui s'occupe du reste.

Séparation des responsabilités financières

- **Abonnement** décide quoi facturer (plans, quotas, overage)
- **Paiement** exécute la transaction via le provider externe
- **Wallet** répartit les fonds (commissions, soldes, crédits)

Aucun de ces services ne doit empiéter sur la responsabilité des deux autres.

3.2 Objectifs

#	Objectif	Indicateur
O1	Abstraction providers	Un seul contrat d'API, quel que soit le provider
O2	Multi-provider	Orange Money, MTN MoMo, Stripe, PayPal au MVP
O3	Idempotence	Aucune double facturation, même en cas de retry
O4	Webhooks normalisés	Tous les providers convergent vers un statut unifié
O5	Traçabilité	Chaque transaction tracée de bout en bout
O6	Réconciliation	Rapprochement automatique provider ↔ transactions internes
O7	Event-driven	Confirmation/échec propagés via RabbitMQ
O8	Multi-devise	Configurable par plateforme

4 Parties Prenantes & Utilisateurs

Rôle	Type	Responsabilité
Product Owner	Interne	Priorisation des fonctionnalités
Équipe Backend	Interne	Développement, intégration providers
Équipe Frontend	Interne	Intégration UI paiement (formulaires, redirections)
DevOps	Interne	Infrastructure, sécurité, monitoring
Admin global AGT	Interne	Configuration providers, réconciliation
Admin plateforme	Interne	Consultation transactions de sa plateforme
Services internes	Interne	Abonnement (déclencheur principal), Wallet (futur)
Utilisateurs finaux	Externe	Paiement via Mobile Money, carte, PayPal

5 Besoins Fonctionnels

5.1 Initiation de Paiement

ID	Fonctionnalité	Description	Priorité
F-PAY-01	Initier un paiement	Créer une transaction avec montant, devise, provider, et référence externe	Urgent
F-PAY-02	Choix du provider	Le demandeur spécifie le provider ou laisse Paiement choisir selon la config plateforme	Urgent
F-PAY-03	URL de paiement	Pour les providers redirect (Stripe, PayPal) : retourner l'URL de checkout	Urgent
F-PAY-04	USSD push	Pour Mobile Money : déclencher le push USSD sur le téléphone du payeur	Urgent
F-PAY-05	Idempotence	Chaque demande porte un `idempotency_key`. Doublon = retour du résultat existant	Urgent
F-PAY-06	Métadonnées libres	Le demandeur attache des `metadata` (subscription_id, order_id, etc.)	Urgent

5.2 Webhooks & Confirmation

ID	Fonctionnalité	Description	Priorité
F-WH-01	Réception webhook	Endpoint par provider pour recevoir les callbacks	Urgent
F-WH-02	Vérification signature	Valider l'authenticité du webhook (HMAC, IP whitelist, etc.)	Urgent
F-WH-03	Normalisation statut	Convertir le statut provider vers le statut interne unifié	Urgent
F-WH-04	Mise à jour transaction	Passer la transaction en `succeeded`, `failed`, ou `expired`	Urgent
F-WH-05	Émission événement	Publier `payment.confirmed`, `payment.failed` ou `payment.cancelled` sur RabbitMQ	Urgent
F-WH-06	Idempotence webhook	Un même webhook reçu N fois ne modifie la transaction qu'une fois	Urgent
F-WH-07	Log brut	Stocker le payload brut du webhook pour audit et debug	Urgent

Normalisation des statuts

Chaque provider a sa propre terminologie. Paiement normalise vers un statut unique :

Statut interne	Stripe	Orange Money	MTN MoMo	PayPal
`pending`	payment_intent.created	INITIATED	PENDING	CREATED
`processing`	payment_intent.processing	PENDING	PENDING	APPROVED
`succeeded`	payment_intent.succeeded	SUCCESS	SUCCESSFUL	COMPLETED
`failed`	payment_intent.payment_failed	FAILED	FAILED	VOIDED
`expired`	—	EXPIRED	EXPIRED	EXPIRED
`cancelled`	payment_intent.canceled	—	—	VOIDED

Machine à états — Transitions autorisées (v1.1)

Toute transition non listée ci-dessous est **interdite**. Le code doit rejeter les transitions invalides avec une erreur 400.

```
[creation] --> pending
pending    --> processing       (provider a accuse reception)
pending    --> succeeded        (confirmation instantanee, ex: carte)
pending    --> failed           (rejet immediat provider)
pending    --> expired          (TTL depasse sans reponse)
pending    --> cancelled        (annulation avant traitement)
processing --> succeeded        (webhook confirmation)
processing --> failed           (webhook echec)
processing --> expired          (TTL depasse en cours de traitement)
```

États terminaux (aucune transition sortante) : `succeeded`, `failed`, `expired`, `cancelled`.
Exception admin : un admin global peut forcer `pending` → `succeeded` ou `pending` → `failed` via `force-status`, uniquement après vérification manuelle. Cette action est loggée dans `transaction_status_history` avec `trigger = admin_manual`.

Modèle PaymentMethod abstrait (v1.1)

Le demandeur d'un paiement spécifie un `payment_method` qui abstrait le type et le provider :

type	provider	Données requises
`mobile_money`	orange_money	`phone_number`
`mobile_money`	mtn_momo	`phone_number`
`card`	stripe	Aucune (redirect vers Stripe Checkout)
`external_wallet`	paypal	Aucune (redirect vers PayPal). Distinct du futur Wallet AGT.

Si le demandeur ne spécifie que le `type` (ex : `mobile_money`) sans préciser le provider, Paiement choisit automatiquement le provider de plus haute priorité pour ce type dans la config plateforme.

5.3 Gestion des Providers

ID	Fonctionnalité	Description	Priorité
F-PRV-01	Configuration provider	Créer/modifier un provider avec credentials chiffrées (AES-256)	**Urgent**
F-PRV-02	Providers par plateforme	Chaque plateforme configure ses providers actifs et leur priorité	**Urgent**
F-PRV-03	Fallback provider	Si le provider principal échoue, tenter le suivant dans l'ordre de priorité	Normale
F-PRV-04	Circuit breaker	Désactiver temporairement un provider en cas d'échecs répétés	Normale
F-PRV-05	Health check provider	Vérifier périodiquement la disponibilité de chaque provider	Normale

Architecture provider — Strategy Pattern

Chaque provider est implémenté comme un **adapter** derrière une interface commune :
— `initiate_payment(amount, currency, metadata)` → `PaymentResult`

— `verify_webhook(headers, payload)` → `bool`
— `normalize_status(raw_status)` → `InternalStatus`
— `get_payment_url()` → `str | None`

Ajouter un nouveau provider = ajouter un adapter, zéro modification du code métier.

5.4 Transactions & Traçabilité

ID	Fonctionnalité	Description	Priorité
F-TRX-01	Détail transaction	Consulter une transaction avec son historique de statuts	**Urgent**
F-TRX-02	Liste transactions	Filtrer par plateforme, statut, provider, date, user	**Urgent**
F-TRX-03	Historique de statuts	Chaque changement de statut est loggé avec timestamp	**Urgent**
F-TRX-04	Référence croisée	Chaque transaction porte la référence provider ET la référence interne du demandeur	**Urgent**

5.5 Réconciliation

ID	Fonctionnalité	Description	Priorité
F-REC-01	Réconciliation auto	Cron : comparer les transactions internes avec les relevés providers	**Urgent**
F-REC-02	Détection anomalies	Identifier les transactions payées chez le provider mais non confirmées en interne (et inversement)	**Urgent**
F-REC-03	Rapport de réconciliation	Générer un rapport par provider et par période	Normale
F-REC-04	Résolution manuelle	Interface admin pour résoudre les anomalies	Normale

Scénarios de réconciliation

— **Paiement confirmé chez le provider, pending en interne** : webhook perdu. Le cron de réconciliation interroge l'API du provider et met à jour.
— **Transaction interne succeeded, absente chez le provider** : anomalie critique. Alerte admin immédiate.
— **Montant différent** : anomalie. Transaction flagée pour résolution manuelle.

Fréquence : toutes les 6 heures pour les transactions des dernières 48h.

5.6 Administration

ID	Fonctionnalité	Description	Priorité
F-ADM-01	Stats par plateforme	Volume, montant total, taux de succès, par provider	Normale
F-ADM-02	Stats par provider	Latence, taux d'échec, disponibilité	Normale
F-ADM-03	Forcer un statut	Admin peut forcer `succeeded` ou `failed` après vérification manuelle	**Urgent**
F-ADM-04	Rejouer un webhook	Resoumettre un webhook en cas de traitement échoué	Normale

6 Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Temps d'initiation paiement	$<$ 500ms (P95) hors latence provider
NF-02	Performance	Traitement webhook	$<$ 200ms (P95)
NF-03	Disponibilité	Uptime	99.9%
NF-04	Sécurité	Credentials providers chiffrés	AES-256
NF-05	Sécurité	Webhooks vérifiés	Signature HMAC ou IP whitelist
NF-06	Sécurité	Auth obligatoire	JWT ou S2S
NF-07	Sécurité	PCI-DSS	Aucune donnée carte stockée (délégué à Stripe)
NF-08	Sécurité	Rate limiting	Configurable par endpoint
NF-09	Scalabilité	Horizontal	Stateless, multi-instance
NF-10	Observabilité	Logs structurés	JSON, request ID, masquage données sensibles
NF-11	Observabilité	Health check	`GET /payments/health`
NF-12	Données	Timestamps	TIMESTAMPTZ
NF-13	Traitement	Crons réconciliation	Celery Beat, retry 3x backoff (1min, 5min, 15min), DLQ
NF-14	Fiabilité	Idempotence	Sur initiation ET sur webhook

7 Architecture Technique

7.1 Vue d'ensemble

Le service Paiement est un microservice REST stateless, déployé en conteneur Docker. Il expose une API JSON consommée par les services internes (Abonnement, Wallet) et les plateformes

clientes. Il reçoit les webhooks des providers externes sur des endpoints dédiés.

7.2 Stack Technique

Composant	Technologie
Langage & Framework	Python 3.11+ / Django 5.x
API	Django REST Framework (DRF)
Base de données	PostgreSQL 15+
ORM	Django ORM
Migrations	Django Migrations
Cache	Redis 7+ (idempotence, circuit breaker)
Scheduler	Celery + Celery Beat (réconciliation, relance)
Message Broker	RabbitMQ (événements sortants)
Chiffrement	AES-256 pour credentials providers
Validation	DRF Serializers
Conteneurisation	Docker + Docker Compose
CI/CD	GitHub Actions

7.3 Dépendances & Règles d'Architecture

Service	Type	Usage
Service Auth	Interne	Validation JWT via `GET /auth/verify-token`. Validation S2S via `POST /auth/s2s/introspect`.
Service Users	Interne	Résolution `user_id`, vérification statut actif avant paiement.
Service Abonnement	Interne	Consommateur principal. Émet `subscription.payment_required`. Reçoit `payment.confirmed/failed/cancelled`.
Service Wallet	Interne (futur)	Cash-in/cash-out. Reçoit `payment.confirmed` pour créditer les wallets.
Service Notification	Interne	Alertes : paiement réussi, échoué, reçu de paiement.
Orange Money API	Externe	Provider Mobile Money Orange Cameroun
MTN MoMo API	Externe	Provider Mobile Money MTN Cameroun
Stripe API	Externe	Provider carte bancaire (Visa, Mastercard)
PayPal API	Externe	Provider paiement international

Règle stricte de dépendance

Paiement ne dépend d'aucun service métier. Il est **consommé** par Abonnement et Wallet via RabbitMQ (event-driven) ou par les plateformes via API REST. Paiement ne connaît pas la nature de ce qui est payé : il reçoit un montant, une devise, un provider et des metadata opaques.

Flux principal :

1. Abonnement émet `subscription.payment_required`
2. Paiement consomme l'événement, initie la transaction chez le provider
3. Provider confirme (webhook)
4. Paiement émet `payment.confirmed`
5. Abonnement consomme et active l'abonnement

7.4 Stratégie de résilience

Fallback et circuit breaker

Fallback provider : si le provider principal échoue (timeout, 5xx, indisponibilité), Paiement tente le provider suivant dans l'ordre de priorité configuré par la plateforme. Le fallback ne s'applique qu'aux erreurs techniques, pas aux refus de paiement (fonds insuffisants, carte refusée).
Circuit breaker : si un provider accumule > 50% d'échecs techniques sur les 100 dernières requêtes (fenêtre glissante), il est automatiquement désactivé pour 5 minutes. Après la période de cooldown, un appel de test est envoyé. Si succès, le circuit se referme. État stocké dans Redis.
Webhook retry : si le traitement interne d'un webhook échoue (erreur applicative, DB down), le webhook brut est persisté dans `webhook_logs` et un cron de rattrapage le retraite dans les 15 minutes.

Timeout et expiration par provider (v1.1)

Chaque provider a un comportement temporel différent. Paiement gère les TTL suivants :

Provider	TTL par défaut	Comportement à expiration
Orange Money	5 minutes	Le push USSD expire. Transaction → `expired`.
MTN MoMo	5 minutes	Idem.
Stripe	30 minutes	La session Checkout expire. Transaction → `expired`.
PayPal	60 minutes	Le lien de paiement expire. Transaction → `expired`.

Implémentation : à la création de la transaction, `expires_at` est calculé selon le provider. Un cron Celery Beat (toutes les minutes) passe les transactions `pending` dont `expires_at < now()` en statut `expired` et émet `payment.failed` avec `failure_reason = "expired"`.
Annulation : un utilisateur peut annuler un paiement `pending` via l'API (`POST /payments/{id}/cancel`). La transaction passe en `cancelled`. Si le provider supporte l'annulation, elle est propagée.

7.5 Diagramme d'architecture

```
Abonnement        Wallet (futur)      Plateformes
    |                  |                   |
[RabbitMQ]         [RabbitMQ]           [REST API]
    |                  |                   |
    +--------+--------+-------+-----------+
             |                 |
      Payment Service (API REST + Workers)
        |         |         |         |
    Orange    MTN MoMo  Stripe   PayPal
    Money
        |         |         |         |
        +---[webhooks]----+--------+
                  |
       Payment Service
                  |
      [RabbitMQ: payment.confirmed / failed / cancelled]
```

```
                |
        +-------+-------+
        |               |
    Abonnement       Wallet
```

8 Modélisation

8.1 Diagramme de Classes

Le diagramme de classes complet est fourni en annexe au format PlantUML (voir section 12). Il couvre les entités : `Transaction`, `TransactionStatusHistory`, `WebhookLog`, `ProviderConfig`, `PlatformPaymentConfig` et `ReconciliationReport`.

8.2 MLD — Modèle Logique de Données

8.2.1 Table `transactions`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme (UUID Auth)
user_id	UUID	NULL	Payeur (`users_auth.id`). NULL si paiement S2S sans user.
provider	VARCHAR(30)	NOT NULL	orange_money, mtn_momo, stripe, paypal
provider_tx_id	VARCHAR(255)	NULL	Identifiant de la transaction chez le provider
idempotency_key	UUID	UNIQUE, NOT NULL	Clé d'idempotence fournie par le demandeur
amount	DECIMAL(12,2)	NOT NULL	Montant
currency	VARCHAR(3)	NOT NULL	Devise (XAF, EUR, USD)
status	VARCHAR(20)	NOT NULL	pending, processing, succeeded, failed, expired, cancelled
source	VARCHAR(30)	NOT NULL	Origine : subscription, wallet, platform_direct
source_reference_id	UUID	NULL	Référence du demandeur (subscription_id, order_id, etc.)
metadata	JSONB	NULL	Données opaques du demandeur
payment_url	TEXT	NULL	URL de checkout (Stripe, PayPal)
phone_number	VARCHAR(20)	NULL	Numéro pour Mobile Money (masqué dans les logs)
failure_reason	TEXT	NULL	Raison de l'échec (normalisée)
provider_raw_status	VARCHAR(50)	NULL	Statut brut du provider
expires_at	TIMESTAMPTZ	NULL	Expiration de la transaction (si applicable)
confirmed_at	TIMESTAMPTZ	NULL	Date de confirmation
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Index : `(platform_id, status)`, `(user_id)`, `(provider_tx_id)`, `(source, source_reference_id)`.

8.2.2 Table `transaction_status_history`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
transaction_id	UUID	FK → transactions	Transaction
from_status	VARCHAR(20)	NULL	Statut précédent (NULL si création)
to_status	VARCHAR(20)	NOT NULL	Nouveau statut
trigger	VARCHAR(30)	NOT NULL	webhook, cron_reconciliation, admin_manual, api_call
provider_raw	JSONB	NULL	Données brutes du provider au moment du changement
created_at	TIMESTAMPTZ	NOT NULL	Date du changement

8.2.3 Table `webhook_logs`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
provider	VARCHAR(30)	NOT NULL	Provider source
event_type	VARCHAR(50)	NOT NULL	Type d'événement provider
payload	JSONB	NOT NULL	Payload brut (données sensibles masquées)
headers	JSONB	NULL	Headers de la requête
signature_valid	BOOLEAN	NOT NULL	Signature vérifiée
transaction_id	UUID	FK → transactions, NULL	Transaction associée (NULL si non trouvée)
processing_status	VARCHAR(20)	DEFAULT 'pending'	pending, processed, failed, ignored
error_message	TEXT	NULL	Erreur de traitement
created_at	TIMESTAMPTZ	NOT NULL	Date de réception
processed_at	TIMESTAMPTZ	NULL	Date de traitement

Index : `(provider, processing_status)` pour le cron de rattrapage.

8.2.4 Table provider_configs

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
provider	VARCHAR(30)	NOT NULL	orange_money, mtn_momo, stripe, paypal
display_name	VARCHAR(100)	NOT NULL	Nom affiché
is_active	BOOLEAN	DEFAULT true	Provider actif globalement
credentials_encrypted	TEXT	NOT NULL	Credentials chiffrées AES-256
api_base_url	VARCHAR(255)	NOT NULL	URL de base de l'API provider
webhook_secret_encrypted	TEXT	NULL	Secret de vérification webhook (chiffré)
supported_currencies	JSONB	NOT NULL	Devises supportées
config	JSONB	NULL	Config spécifique (timeout, options)
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(provider)`.

8.2.5 Table platform_payment_configs

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme (UUID Auth)
provider	VARCHAR(30)	NOT NULL	Provider
is_active	BOOLEAN	DEFAULT true	Actif pour cette plateforme
priority	INT	DEFAULT 0	Ordre de priorité (fallback)
default_currency	VARCHAR(3)	DEFAULT 'XAF'	Devise par défaut
config_overrides	JSONB	NULL	Surcharges de config spécifiques à la plateforme
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(platform_id, provider)`.

8.2.6 Table reconciliation_reports

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
provider	VARCHAR(30)	NOT NULL	Provider concerné
period_start	TIMESTAMPTZ	NOT NULL	Début de la période
period_end	TIMESTAMPTZ	NOT NULL	Fin de la période
total_internal	INT	NOT NULL	Nombre de transactions internes
total_provider	INT	NOT NULL	Nombre de transactions provider
matched	INT	NOT NULL	Transactions concordantes
mismatched	INT	NOT NULL	Anomalies détectées
missing_internal	INT	NOT NULL	Présentes chez provider, absentes en interne
missing_provider	INT	NOT NULL	Présentes en interne, absentes chez provider
details	JSONB	NULL	Détail des anomalies
status	VARCHAR(20)	NOT NULL	completed, failed
created_at	TIMESTAMPTZ	NOT NULL	Date du rapport

9 Contrats d'API

Base URL : `/api/v1`. Tous les endpoints retournent du JSON. Les endpoints protégés requièrent `Authorization: Bearer <token>`.

9.1 Health Check

9.1.1 GET /payments/health

Vérifie l'état du service. Aucune authentification requise.

Response 200 :

```
{
  "status": "healthy",
  "database": "ok",
  "redis": "ok",
  "rabbitmq": "ok",
  "version": "1.2.0"
}
```

9.2 Initiation de Paiement

9.2.1 POST /payments/initiate

Initie une transaction. Auth : Bearer token ou S2S.

Request body :

```
{
  "platform_id": "uuid",
  "user_id": "user-uuid",
  "provider": "orange_money",
  "amount": 15000,
  "currency": "XAF",
  "source": "subscription",
```

```
  "source_reference_id": "sub-uuid",
  "idempotency_key": "unique-uuid",
  "phone_number": "+237600000000",
  "metadata": {
    "subscription_id": "sub-uuid",
    "plan_name": "Pro",
    "reason": "renewal"
  }
}
```

Response 201 (Mobile Money — USSD push déclenché) :

```
{
  "transaction_id": "tx-uuid",
  "status": "pending",
  "provider": "orange_money",
  "amount": 15000,
  "currency": "XAF",
  "message": "USSD push sent, waiting for user confirmation"
}
```

Response 201 (Stripe/PayPal — URL de checkout) :

```
{
  "transaction_id": "tx-uuid",
  "status": "pending",
  "provider": "stripe",
  "amount": 15000,
  "currency": "XAF",
  "payment_url": "https://checkout.stripe.com/pay/cs_xxx",
  "message": "Redirect user to payment_url"
}
```

Response 200 (idempotence — transaction déjà créée) :

```
{
  "transaction_id": "tx-uuid-existing",
  "status": "pending",
  "message": "Idempotent request, returning existing transaction"
}
```

Codes : 201 Created, 200 Idempotent hit, 400 Validation error, 401 Unauthorized, 404 Provider not found, 422 Provider rejected, 503 Provider unavailable.

Contrat d'idempotence (v1.1)

Règle contractuelle : le champ `idempotency_key` est **obligatoire** sur `POST /payments/initiate`. C'est une contrainte `UNIQUE NOT NULL` en base.

Comportement :

- Première requête avec une clé donnée → 201 Created, transaction créée
- Requête identique (même clé) → 200 OK, transaction existante retournée sans modification ni double envoi au provider
- Requête avec même clé mais payload différent → 409 Conflict

Responsabilité du demandeur : générer un UUID v4 unique par intention de paiement. Le même UUID doit être renvoyé en cas de retry réseau. Cela couvre : double-clic frontend, retry RabbitMQ, timeout HTTP avec relance.

TTL : les clés d'idempotence sont conservées en base indéfiniment (liées à la transaction). Pas de TTL : une transaction existe pour toujours.

9.2.2 GET /payments/{transactionId}

Détail d'une transaction avec historique de statuts.

Response 200 :

```
{
  "id": "tx-uuid",
  "platform_id": "uuid",
  "user_id": "user-uuid",
  "provider": "orange_money",
  "amount": 15000,
  "currency": "XAF",
  "status": "succeeded",
  "source": "subscription",
  "source_reference_id": "sub-uuid",
  "provider_tx_id": "OM-123456",
  "confirmed_at": "2026-04-03T12:05:00Z",
  "created_at": "2026-04-03T12:00:00Z",
  "status_history": [
    { "from": null, "to": "pending", "trigger": "api_call", "at": "..." },
    { "from": "pending", "to": "succeeded", "trigger": "webhook", "at": "..." }
  ]
}
```

Codes : 200, 404.

9.2.3 GET /payments

Liste des transactions. Query params : `?platform_id&user_id&status&provider&source&from_date&to_date&`

Codes : 200, 401.

9.2.4 POST /payments/{transactionId}/cancel (v1.1)

Annulation d'un paiement `pending`. Auth : Bearer token ou S2S.

Response 200 :

```
{
  "transaction_id": "tx-uuid",
  "status": "cancelled",
  "message": "Payment cancelled"
}
```

Codes : 200, 400 Transaction not in pending state, 404.

9.3 Webhooks Providers

9.3.1 POST /payments/webhooks/orange-money

9.3.2 POST /payments/webhooks/mtn-momo

9.3.3 POST /payments/webhooks/stripe

9.3.4 POST /payments/webhooks/paypal

Chaque endpoint reçoit le callback du provider. **Aucune authentification JWT** : la vérification se fait par signature HMAC ou IP whitelist selon le provider.

Comportement commun :

1. Vérifier la signature du webhook
2. Stocker le payload brut dans `webhook_logs`
3. Identifier la transaction via `provider_tx_id`
4. Normaliser le statut
5. Mettre à jour la transaction si le statut est un état terminal (`succeeded`, `failed`)
6. Émettre l'événement RabbitMQ correspondant

Response : 200 OK (toujours, même en cas d'erreur interne, pour éviter les retries provider inutiles). Les erreurs sont gérées en interne via le cron de rattrapage.

Sécurité des webhooks (v1.1)

4 couches de protection :

1. **Signature HMAC** : chaque provider signe ses webhooks (Stripe : `Stripe-Signature` HMAC-SHA256, Orange/MTN : header propriétaire). Le secret est stocké chiffré dans `provider_configs.webhook_secret_encrypted`. Toute signature invalide → log + rejet silencieux (200 OK retourné pour ne pas révéler l'erreur).
2. **IP whitelist** : pour les providers Mobile Money qui ne signent pas systématiquement, filtrage par IP source. Liste configurable dans `provider_configs.config`.
3. **Replay protection** : chaque webhook porte un identifiant unique (ex : `event_id` Stripe). Paiement stocke les IDs traités dans Redis (TTL 72h). Tout doublon est ignoré.
4. **Timestamp validation** : si le webhook contient un timestamp, rejeter s'il date de plus de 5 minutes (protection contre les attaques par replay différé).

9.4 Configuration Providers

9.4.1 POST /payments/providers

Création d'un provider. Auth : admin global.

Request body :

```
{
  "provider": "orange_money",
  "display_name": "Orange Money Cameroun",
  "credentials": { "api_key": "xxx", "api_secret": "yyy" },
  "api_base_url": "https://api.orange.com/...",
  "webhook_secret": "whsec_xxx",
  "supported_currencies": ["XAF"]
}
```

Codes : 201, 400, 409 Provider already exists.

Note : `credentials` et `webhook_secret` sont chiffrés AES-256 avant stockage. Le secret brut n'est jamais retourné.

9.4.2 PUT /payments/providers/{provider}

Modification. Codes : 200, 400, 404.

9.4.3 PUT /payments/platforms/{platformId}/providers

Configuration des providers pour une plateforme (priorité, devise, overrides).

Request body :

```
{
  "providers": [
    { "provider": "orange_money", "priority": 1, "is_active": true },
    { "provider": "mtn_momo", "priority": 2, "is_active": true },
    { "provider": "stripe", "priority": 3, "is_active": true }
  ],
  "default_currency": "XAF"
}
```

Codes : 200, 400, 403.

9.5 Administration

9.5.1 GET /payments/admin/stats

Query params : `?platform_id&provider&from_date&to_date`

Response 200 :

```
{
  "total_transactions": 12450,
  "total_amount": 185000000,
  "currency": "XAF",
  "success_rate": 94.2,
  "by_provider": [
    { "provider": "orange_money", "count": 8200, "success_rate": 95.1 },
    { "provider": "stripe", "count": 3100, "success_rate": 93.8 },
    { "provider": "mtn_momo", "count": 1150, "success_rate": 91.5 }
  ]
}
```

Codes : 200, 403.

9.5.2 POST /payments/admin/{transactionId}/force-status

Forcer un statut après vérification manuelle. Auth : admin global.

Request body :

```
{
  "status": "succeeded",
  "reason": "Verified manually via Orange Money dashboard"
}
```

Codes : 200, 400 Invalid status transition, 403, 404.

9.5.3 GET /payments/admin/reconciliation

Liste des rapports de réconciliation. Query params : `?provider&from_date&to_date`.

Codes : 200, 403.

9.5.4 DELETE /payments/by-user/{userId}

Purge RGPD. Auth : S2S token. Anonymise les données personnelles des transactions (phone_number, user_id) sans supprimer les transactions (obligation comptable).

Codes : 200, 401, 404.

10 Contrats Inter-Services

10.1 Paiement → Auth (JWT utilisateur)

Paramètre	Valeur
Endpoint	`GET /auth/verify-token`
Auth	`Authorization: Bearer <access_token>`
Données extraites	`user_id` (`sub`), `platform_id`
Cache	Redis TTL 30s
Comportement échec	Retourner 401 Unauthorized

10.2 Paiement → Auth (token S2S)

Paramètre	Valeur
Endpoint	`POST /auth/s2s/introspect`
Données	`client_id`, `service_name`
Usage	Appels S2S entrants (Abonnement déclenchant un paiement)

10.3 Abonnement → Paiement (événements RabbitMQ entrants)

Événement consommé	Action Paiement
`subscription.payment_required`	Initier une transaction. Payload contient : `idempotency_key`, `subscription_id`, `amount`, `currency`, `subscriber_type`, `subscriber_id`, `reason`. Paiement résout le provider via la config plateforme.
`subscription.overage_billing`	Initier une transaction pour facturation overage. Payload contient : `idempotency_key`, `quota_snapshot`.

Idempotence événements entrants

Paiement déduplique sur `idempotency_key`. Si une transaction existe déjà pour cette clé, l'événement est ignoré (log info). Cela couvre les retries RabbitMQ et les doubles émissions.

10.4 Paiement → Abonnement / Wallet (événements RabbitMQ sortants)

Événement publié	Payload
`payment.confirmed`	`event_id` (UUID), `payment_reference_id` (= transaction.id), `idempotency_key`, `amount`, `currency`, `provider`, `source`, `source_reference_id`, `metadata`, `confirmed_at`
`payment.failed`	`event_id`, `payment_reference_id`, `idempotency_key`, `amount`, `currency`, `provider`, `source`, `source_reference_id`, `failure_reason`, `failed_at`
`payment.cancelled`	`event_id`, `payment_reference_id`, `idempotency_key`, `source`, `source_reference_id`, `cancelled_at`
`payment.cancelled`	`event_id`, `payment_reference_id`, `idempotency_key`, `source`, `source_reference_id`, `cancelled_at`

Routage des événements

Les événements sont publiés sur un exchange RabbitMQ de type `topic`. Les consommateurs s'abonnent selon leur besoin :

- Abonnement écoute `payment.confirmed` et `payment.cancelled` où `source = subscription`
- Wallet écoutera `payment.confirmed` où `source = wallet`
- Notification écoute `payment.confirmed`, `payment.failed` et `payment.cancelled` pour envoyer les alertes

10.5 Paiement → Notification (alertes)

Événement	Notification déclenchée
Transaction succeeded	Email/push : « Paiement de X FCFA reçu » + reçu
Transaction failed	Email/push : « Échec du paiement, veuillez réessayer »
Transaction cancelled	Push : « Votre paiement a été annulé »
Transaction expired	Push : « Votre paiement a expiré »
Transaction cancelled	Push : « Votre paiement a été annulé »

10.6 Users → Paiement (Purge RGPD)

Paramètre	Valeur
Endpoint	`DELETE /api/v1/payments/by-user/{userId}`
Auth	Token S2S
Usage	Appelé par Users dans le workflow de purge RGPD
Données anonymisées	`user_id` → NULL, `phone_number` → NULL. Transactions conservées (obligation comptable).
Séquence	Avant Auth purge

RGPD — obligation comptable

Les transactions ne sont **jamais supprimées** physiquement. La purge RGPD **anonymise** les données personnelles (`user_id`, `phone_number`) tout en conservant les montants, dates et références pour la traçabilité comptable légale.

11 Matrice d'Autorisation

Endpoint	JWT	S2S	Admin Plat.	Admin Global
POST /payments/initiate	✓	✓		
GET /payments/{id}	✓	✓	✓	✓
GET /payments	✓	✓	✓	✓
POST /payments/{id}/cancel	✓	✓		✓
POST /payments/webhooks/*		*Signature provider (pas de JWT)*		
POST /payments/providers				✓
PUT /payments/providers/{provider}				✓
PUT /payments/platforms/{platformId}/providers			✓	✓
GET /payments/admin/stats			✓	✓
POST /payments/admin/{id}/force-status				✓
GET /payments/admin/reconciliation				✓
DELETE /payments/by-user/{userId}		✓		✓

12 Recommandations & Évolutions

- **Remboursements (phase 2)** : endpoint `POST /payments/{id}/refund` avec montant partiel ou total, propagé vers le provider.
- **Nouveaux providers** : CinetPay, Campay, Flutterwave via le pattern adapter.
- **Paiement récurrent natif** : certains providers (Stripe) supportent la récurrence native. Possibilité de déléguer la récurrence au provider pour réduire la charge sur Abonnement.
- **3D Secure** : gestion du challenge d'authentification carte.
- **Monitoring** : métriques Prometheus (transactions/s, latence par provider, taux de succès, circuit breaker state).
- **Versioning API** : préfixe `/api/v1`. Breaking changes → `/api/v2`.
- **Dashboard financier** : vue consolidée cross-plateforme pour la direction.

Annexes

Glossaire

Terme	Définition
Provider	Fournisseur de paiement externe (Orange Money, Stripe, etc.)
Transaction	Unité de paiement tracée de l'initiation à la confirmation
Webhook	Callback HTTP envoyé par le provider pour notifier un changement de statut
Idempotence	Garantie qu'une opération exécutée N fois produit le même résultat
Circuit breaker	Mécanisme de protection qui désactive un provider défaillant temporairement
Réconciliation	Rapprochement entre les transactions internes et les relevés du provider
USSD push	Notification envoyée sur le téléphone du payeur pour confirmer un paiement Mobile Money
PCI-DSS	Norme de sécurité pour le traitement des données de carte bancaire
DLQ	Dead Letter Queue — file d'attente pour les messages non traités
S2S	Service-to-Service — communication entre microservices

Code PlantUML — Diagramme de Classes

Le code PlantUML ci-dessous correspond au diagramme de classes complet du service Paiement v1.2. Il peut être rendu sur `plantuml.com` ou tout outil compatible.

```
@startuml Payment Service - Class Diagram v1.2

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "transactions" as tx {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  user_id : UUID
  * provider : VARCHAR(30)
  provider_tx_id : VARCHAR(255)
  * idempotency_key : UUID <<UQ>>
  * amount : DECIMAL(12,2)
  * currency : VARCHAR(3)
  * status : VARCHAR(20)
  * source : VARCHAR(30)
  source_reference_id : UUID
  metadata : JSONB
  payment_url : TEXT
  phone_number : VARCHAR(20)
  failure_reason : TEXT
  provider_raw_status : VARCHAR(50)
  expires_at : TIMESTAMPTZ
  confirmed_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

entity "transaction_status_history" as history {
  * id : UUID <<PK>>
  --
  * transaction_id : UUID <<FK>>
  from_status : VARCHAR(20)
  * to_status : VARCHAR(20)
  * trigger : VARCHAR(30)
  provider_raw : JSONB
  * created_at : TIMESTAMPTZ
}

entity "webhook_logs" as whlogs {
  * id : UUID <<PK>>
  --
  * provider : VARCHAR(30)
  * event_type : VARCHAR(50)
  * payload : JSONB
  headers : JSONB
  * signature_valid : BOOLEAN
  transaction_id : UUID <<FK>>
  processing_status : VARCHAR(20) = 'pending'
  error_message : TEXT
  * created_at : TIMESTAMPTZ
  processed_at : TIMESTAMPTZ
}

entity "provider_configs" as providers {
  * id : UUID <<PK>>
  --
  * provider : VARCHAR(30) <<UQ>>
  * display_name : VARCHAR(100)
  is_active : BOOLEAN = true
  * credentials_encrypted : TEXT
  * api_base_url : VARCHAR(255)
  webhook_secret_encrypted : TEXT
  * supported_currencies : JSONB
  config : JSONB
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
```

```
}

entity "platform_payment_configs" as platconfigs {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  * provider : VARCHAR(30)
  is_active : BOOLEAN = true
  priority : INT = 0
  default_currency : VARCHAR(3) = 'XAF'
  config_overrides : JSONB
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (platform_id, provider)
}

entity "reconciliation_reports" as recon {
  * id : UUID <<PK>>
  --
  * provider : VARCHAR(30)
  * period_start : TIMESTAMPTZ
  * period_end : TIMESTAMPTZ
  * total_internal : INT
  * total_provider : INT
  * matched : INT
  * mismatched : INT
  * missing_internal : INT
  * missing_provider : INT
  details : JSONB
  * status : VARCHAR(20)
  * created_at : TIMESTAMPTZ
}

tx ||--o{ history    : "1..* status changes"
tx ||--o{ whlogs     : "0..* webhook logs"

@enduml
```